



CELEBAL
TECHNOLOGIES

Celebal Summer Internship 2026

Week 2 Task: E-Commerce Sales Database

1. Scenario & Business Context

You are a Junior Data Analyst at ShopEase, a mid-sized e-commerce company that sells electronics, clothing, and home products across India. The management team wants to understand sales patterns, customer behavior, and product performance to make better business decisions.

You have been given access to the company's relational database containing information about customers, products, orders, and order details. Your task is to write SQL queries to extract meaningful insights from this data.

⚠ Note: You may use MySQL, PostgreSQL, or any SQL-compatible RDBMS to complete these tasks. All queries should follow standard SQL syntax.

2. Database Schema & Constraints

The database consists of 4 tables. Study the schema below carefully — pay attention to Primary Keys, Foreign Keys, and Constraints.

Table: customers

```
CREATE TABLE customers (  
  customer_id INT PRIMARY KEY,  
  first_name VARCHAR(50) NOT NULL,  
  last_name VARCHAR(50) NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  city VARCHAR(50) NOT NULL,  
  state VARCHAR(50) NOT NULL,  
  join_date DATE NOT NULL,  
  is_premium BOOLEAN DEFAULT FALSE  
);  
  
-- Index for filtering by city/state  
CREATE INDEX idx_customers_city ON customers(city);  
CREATE INDEX idx_customers_state ON customers(state);
```

Table: products

```
CREATE TABLE products (  
  product_id INT PRIMARY KEY,  
  product_name VARCHAR(100) NOT NULL,  
  category VARCHAR(50) NOT NULL,  
  brand VARCHAR(50) NOT NULL,  
  unit_price DECIMAL(10,2) NOT NULL CHECK (unit_price > 0),  
  stock_qty INT NOT NULL DEFAULT 0 CHECK (stock_qty >= 0)  
);  
  
-- Index for filtering by category  
CREATE INDEX idx_products_category ON products(category);
```

Table: orders

```
CREATE TABLE orders (  
  order_id INT PRIMARY KEY,  
  customer_id INT NOT NULL,  
  order_date DATE NOT NULL,  
  status VARCHAR(20) NOT NULL DEFAULT 'Pending'
```

```

        CHECK (status IN ('Pending','Shipped','Delivered','Cancelled')),
        total_amount DECIMAL(12,2) NOT NULL CHECK (total_amount >= 0),

        FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
    );

-- Index for date-based filtering and sorting
CREATE INDEX idx_orders_date ON orders(order_date);
CREATE INDEX idx_orders_status ON orders(status);

```

Table: order_items

```

CREATE TABLE order_items (
    item_id          INT          PRIMARY KEY,
    order_id         INT          NOT NULL,
    product_id       INT          NOT NULL,
    quantity         INT          NOT NULL CHECK (quantity > 0),
    unit_price       DECIMAL(10,2) NOT NULL CHECK (unit_price > 0),
    discount_pct     DECIMAL(5,2) DEFAULT 0 CHECK (discount_pct BETWEEN 0 AND 100),

    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);

```

Entity Relationships

```

customers  —(1:N)—>  orders
orders     —(1:N)—>  order_items
products   —(1:N)—>  order_items

customers.customer_id  <—FK—  orders.customer_id
orders.order_id        <—FK—  order_items.order_id
products.product_id    <—FK—  order_items.product_id

```

3. Sample Data

Below is sample data to help you understand the tables. Use the provided INSERT statements to load data into your database.

customers (sample rows)

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	TRUE
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	FALSE
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	TRUE
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	FALSE
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	TRUE
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	FALSE
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	TRUE
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	FALSE

products (sample rows)

product_id	product_name	category	brand	unit_price	stock_qty
201	Wireless Earbuds	Electronics	BoAt	1499.0	250
202	Cotton T-Shirt	Clothing	Levi's	799.0	500
203	Smart Watch	Electronics	Noise	2999.0	150
204	Running Shoes	Clothing	Nike	4599.0	120
205	Bluetooth Speaker	Electronics	JBL	3499.0	200
206	Bedsheet Set	Home	Spaces	1299.0	300
207	Laptop Stand	Electronics	AmazonBasics	899.0	180
208	Cushion Covers (Set)	Home	HomeCenter	599.0	400

orders (sample rows)

order_id	customer_id	order_date	status	total_amount
1001	101	2024-08-01	Delivered	4498.0
1002	102	2024-08-03	Delivered	799.0
1003	103	2024-08-05	Shipped	7498.0
1004	101	2024-08-10	Delivered	3499.0
1005	104	2024-08-12	Cancelled	2999.0
1006	105	2024-08-15	Delivered	5898.0
1007	106	2024-08-18	Pending	1299.0
1008	103	2024-08-20	Delivered	899.0
1009	107	2024-08-25	Shipped	6098.0
1010	108	2024-08-28	Delivered	1598.0

order_items (sample rows)

item_id	order_id	product_id	quantity	unit_price	discount_pct
5001	1001	201	2	1499.0	0
5002	1001	207	1	899.0	10
5003	1002	202	1	799.0	0
5004	1003	203	1	2999.0	0
5005	1003	204	1	4599.0	5
5006	1004	205	1	3499.0	0
5007	1005	203	1	2999.0	0
5008	1006	201	1	1499.0	10
5009	1006	204	1	4599.0	5
5010	1007	206	1	1299.0	0
5011	1008	207	1	899.0	0
5012	1009	205	1	3499.0	0
5013	1009	208	2	599.0	15

5014	1010	206	1	1299.0	0
5015	1010	208	1	599.0	0

Data Loading — INSERT Statements

Copy and execute the following SQL to load sample data into your database:

```
-- ===== INSERT: customers =====
INSERT INTO customers VALUES
(101, 'Aarav', 'Sharma', 'aarav.s@email.com', 'Mumbai', 'Maharashtra', '2024-01-15',
TRUE),
(102, 'Priya', 'Patel', 'priya.p@email.com', 'Ahmedabad', 'Gujarat', '2024-02-20',
FALSE),
(103, 'Rohan', 'Gupta', 'rohan.g@email.com', 'Delhi', 'Delhi', '2024-03-10',
TRUE),
(104, 'Sneha', 'Reddy', 'sneha.r@email.com', 'Hyderabad', 'Telangana', '2024-04-05',
FALSE),
(105, 'Vikram', 'Singh', 'vikram.s@email.com', 'Jaipur', 'Rajasthan', '2024-05-12',
TRUE),
(106, 'Ananya', 'Iyer', 'ananya.i@email.com', 'Chennai', 'Tamil Nadu', '2024-06-18',
FALSE),
(107, 'Karan', 'Mehta', 'karan.m@email.com', 'Pune', 'Maharashtra', '2024-07-22',
TRUE),
(108, 'Divya', 'Nair', 'divya.n@email.com', 'Kochi', 'Kerala', '2024-08-30',
FALSE);

-- ===== INSERT: products =====
INSERT INTO products VALUES
(201, 'Wireless Earbuds', 'Electronics', 'BoAt', 1499.00, 250),
(202, 'Cotton T-Shirt', 'Clothing', 'Levis', 799.00, 500),
(203, 'Smart Watch', 'Electronics', 'Noise', 2999.00, 150),
(204, 'Running Shoes', 'Clothing', 'Nike', 4599.00, 120),
(205, 'Bluetooth Speaker', 'Electronics', 'JBL', 3499.00, 200),
(206, 'Bedsheet Set', 'Home', 'Spaces', 1299.00, 300),
(207, 'Laptop Stand', 'Electronics', 'AmazonBasics', 899.00, 180),
(208, 'Cushion Covers (Set)', 'Home', 'HomeCenter', 599.00, 400);

-- ===== INSERT: orders =====
INSERT INTO orders VALUES
(1001, 101, '2024-08-01', 'Delivered', 4498.00),
(1002, 102, '2024-08-03', 'Delivered', 799.00),
(1003, 103, '2024-08-05', 'Shipped', 7498.00),
(1004, 101, '2024-08-10', 'Delivered', 3499.00),
(1005, 104, '2024-08-12', 'Cancelled', 2999.00),
(1006, 105, '2024-08-15', 'Delivered', 5898.00),
(1007, 106, '2024-08-18', 'Pending', 1299.00),
(1008, 103, '2024-08-20', 'Delivered', 899.00),
(1009, 107, '2024-08-25', 'Shipped', 6098.00),
(1010, 108, '2024-08-28', 'Delivered', 1598.00);

-- ===== INSERT: order_items =====
INSERT INTO order_items VALUES
(5001, 1001, 201, 2, 1499.00, 0),
(5002, 1001, 207, 1, 899.00, 10),
(5003, 1002, 202, 1, 799.00, 0),
(5004, 1003, 203, 1, 2999.00, 0),
(5005, 1003, 204, 1, 4599.00, 5),
(5006, 1004, 205, 1, 3499.00, 0),
```

```
(5007, 1005, 203, 1, 2999.00, 0),  
(5008, 1006, 201, 1, 1499.00, 10),  
(5009, 1006, 204, 1, 4599.00, 5),  
(5010, 1007, 206, 1, 1299.00, 0),  
(5011, 1008, 207, 1, 899.00, 0),  
(5012, 1009, 205, 1, 3499.00, 0),  
(5013, 1009, 208, 2, 599.00, 15),  
(5014, 1010, 206, 1, 1299.00, 0),  
(5015, 1010, 208, 1, 599.00, 0);
```

Section A — SQL Basics (SELECT, Constraints, Primary Keys)

These questions test your understanding of basic data retrieval, table structure, and database constraints.

Q1. Write a query to display all columns and rows from the customer's table.

USE shopease;

SELECT * FROM customers;

	customer_id	first_name	last_name	email	city	state	join_date	is_premium
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
★	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q2. Retrieve only the first_name, last_name, and city of all customers.

SELECT first_name, last_name, city

FROM customers;

	first_name	last_name	city
▶	Aarav	Sharma	Mumbai
	Priya	Patel	Ahmedabad
	Rohan	Gupta	Delhi
	Sneha	Reddy	Hyderabad
	Vikram	Singh	Jaipur
	Ananya	Iyer	Chennai
	Karan	Mehta	Pune
	Divya	Nair	Kochi

Q3. List all unique categories available in the products table.

SELECT DISTINCT category

FROM products;

	category
▶	Electronics
	Clothing
	Home

Q4. Identify the Primary Key of each table in the schema. Explain why a Primary Key must be unique and NOT NULL.

Table	Primary Key
customers	customer_id
products	product_id
orders	order_id
order_items	item_id

A Primary Key uniquely identifies each row and cannot contain NULL values

Q5. What constraints are applied to the email column in the customers table? What would happen if you tried to insert a duplicate email?

Constraints – UNIQUE and NOT NULL

If we insert the same email twice, MySQL gives a **Duplicate Entry** error

Q6. Try inserting a product with unit_price = -50. What happens and which constraint prevents it? Write both the INSERT statement and explain the error.

We get an error because of CHECK(unit_price > 0)

```
33 23:36:30 INSERT INTO products VALUES (209,'Pen','Stationery','Cello',-50,100) Error Code: 3819. Check constraint 'products_chk_1' is violated.
```


Section B — Filtering & Optimization (WHERE, Indexes)

These questions test your ability to filter data effectively and understand how indexes improve query performance.

Q7. Retrieve all orders with status = 'Delivered'.

	order_id	customer_id	order_date	status	total_amount
▶	1001	101	2024-08-01	Delivered	4498.00
	1002	102	2024-08-03	Delivered	799.00
	1004	101	2024-08-10	Delivered	3499.00
	1006	105	2024-08-15	Delivered	5898.00
	1008	103	2024-08-20	Delivered	899.00
	1010	108	2024-08-28	Delivered	1598.00
•	NULL	NULL	NULL	NULL	NULL

Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.

	product_id	product_name	category	brand	unit_price	stock_qty
▶	203	Smart Watch	Electronics	Noise	2999.00	150
	205	Bluetooth Speaker	Electronics	JBL	3499.00	200
•	NULL	NULL	NULL	NULL	NULL	NULL

Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.

	customer_id	first_name	last_name	email	city	state	join_date	is_premium
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
•	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.

	order_id	customer_id	order_date	status	total_amount
▶	1004	101	2024-08-10	Delivered	3499.00
	1006	105	2024-08-15	Delivered	5898.00
	1007	106	2024-08-18	Pending	1299.00
	1008	103	2024-08-20	Delivered	899.00
	1009	107	2024-08-25	Shipped	6098.00
•	NULL	NULL	NULL	NULL	NULL

Q11. Explain what the index idx_orders_date does. How would it improve the performance of a query that filters orders by order_date? Write a sample query that would benefit from this index.

The idx_orders_date index helps MySQL find rows by order_date faster instead of scanning the entire table.

SELECT *

FROM orders

WHERE order_date = '2024-08-15';

Q12. If you run: `SELECT * FROM customers WHERE YEAR(join_date) = 2024;` — would the index on `join_date` be used? Explain why or why not, and rewrite the query to be index-friendly (SARGable).

No, the index would generally **not** be used efficiently with `YEAR(join_date) = 2024` because applying the `YEAR()` function to the column prevents MySQL from using the index effectively.

Section C — Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)

These questions test your ability to summarize and aggregate data.

Q13. Count the total number of orders in the orders table.

	total_orders
▶	10

Q14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders.

	total_revenue
▶	17191.00

Q15. Calculate the average unit_price of products in each category.

	category	average_price
▶	Electronics	2224.000000
	Clothing	2699.000000
	Home	949.000000

Q16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order.

	status	total_orders	total_revenue
▶	Delivered	6	17191.00
	Shipped	2	13596.00
	Cancelled	1	2999.00
	Pending	1	1299.00

Q17. Find the most expensive (MAX) and cheapest (MIN) product in each category.

	category	highest_price	lowest_price
▶	Electronics	3499.00	899.00
	Clothing	4599.00	799.00
	Home	1299.00	599.00

Q18. List all product categories where the average unit_price is greater than ₹2000. (Hint: Use HAVING clause)

	category	average_price
▶	Electronics	2224.000000
	Clothing	2699.000000

Section D — Joins & Relationships

These questions test your ability to combine data from multiple tables using different types of JOINS.

Q19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name. Show: order_id, order_date, first_name, last_name, total_amount.

	order_id	order_date	first_name	last_name	total_amount
▶	1001	2024-08-01	Aarav	Sharma	4498.00
	1004	2024-08-10	Aarav	Sharma	3499.00
	1002	2024-08-03	Priya	Patel	799.00
	1003	2024-08-05	Rohan	Gupta	7498.00
	1008	2024-08-20	Rohan	Gupta	899.00
	1005	2024-08-12	Sneha	Reddy	2999.00
	1006	2024-08-15	Vikram	Singh	5898.00
	1007	2024-08-18	Ananya	Iyer	1299.00
	1009	2024-08-25	Karan	Mehta	6098.00
	1010	2024-08-28	Divya	Nair	1598.00

Q20. Using a LEFT JOIN, list ALL customers and their orders (if any). Customers with no orders should still appear with NULL values for order columns.

	customer_id	first_name	last_name	order_id	order_date
▶	101	Aarav	Sharma	1001	2024-08-01
	101	Aarav	Sharma	1004	2024-08-10
	102	Priya	Patel	1002	2024-08-03
	103	Rohan	Gupta	1003	2024-08-05
	103	Rohan	Gupta	1008	2024-08-20
	104	Sneha	Reddy	1005	2024-08-12
	105	Vikram	Singh	1006	2024-08-15
	106	Ananya	Iyer	1007	2024-08-18
	107	Karan	Mehta	1009	2024-08-25
	108	Divya	Nair	1010	2024-08-28

Q21. Write a query using JOINS across three tables (orders → order_items → products) to show: order_id, product_name, quantity, unit_price, and discount_pct for each order item.

order_id	product_name	quantity	unit_price	discount_pct
1001	Wireless Earbuds	2	1499.00	0.00
1006	Wireless Earbuds	1	1499.00	10.00
1002	Cotton T-Shirt	1	799.00	0.00
1003	Smart Watch	1	2999.00	0.00
1005	Smart Watch	1	2999.00	0.00
1003	Running Shoes	1	4599.00	5.00
1006	Running Shoes	1	4599.00	5.00
1004	Bluetooth Speaker	1	3499.00	0.00
1009	Bluetooth Speaker	1	3499.00	0.00
1007	Bedsheet Set	1	1299.00	0.00
1010	Bedsheet Set	1	1299.00	0.00
1001	Laptop Stand	1	899.00	10.00
1008	Laptop Stand	1	899.00	0.00
1009	Cushion Covers (...)	2	599.00	15.00
1010	Cushion Covers (...)	1	599.00	0.00

Q22. Explain the difference between LEFT JOIN and RIGHT JOIN with an example from this schema. When would you use a FULL OUTER JOIN?

LEFT JOIN - Returns all rows from the **left** table.

-Matching rows are returned from the right table

```
SELECT c.customer_id,  
       c.first_name,  
       o.order_id  
FROM customers c  
LEFT JOIN orders o  
ON c.customer_id = o.customer_id;
```

RIGHT JOIN - Returns all rows from the **right** table.

-Matching rows are returned from the left table.

```
SELECT c.customer_id,  
       c.first_name,  
       o.order_id  
FROM customers c  
RIGHT JOIN orders o  
ON c.customer_id = o.customer_id;
```

Q23. Identify all Foreign Key relationships in the schema. Explain what would happen if you tried to insert an order with customer_id = 999 (which doesn't exist in customers).

orders.customer_id -> customers.customer_id

order_items.order_id -> orders.order_id

order_items.product_id -> products.product_id

The query **fails** because customer_id = 999 does **not** exist in the customers table.

Section E — Advanced Concepts (CASE, ACID, Transactions)

These questions test your understanding of conditional logic, database reliability principles, and transaction management.

Q24. Write a query using CASE to classify products into price tiers:

- 'Budget' → unit_price < 1000
- 'Mid-Range' → unit_price BETWEEN 1000 AND 3000
- 'Premium' → unit_price > 3000

Display: product_name, unit_price, price_tier.

	product_name	unit_price	price_tier
▶	Wireless Earbuds	1499.00	Mid-Range
	Cotton T-Shirt	799.00	Budget
	Smart Watch	2999.00	Mid-Range
	Running Shoes	4599.00	Premium
	Bluetooth Speaker	3499.00	Premium
	Bedsheet Set	1299.00	Mid-Range
	Laptop Stand	899.00	Budget
	Cushion Covers (Set)	599.00	Budget

Q25. Using a CASE statement inside an aggregate function, count how many orders are 'Delivered' vs 'Not Delivered' (all other statuses). Display the result in a single row.

	Delivered	Not_Delivered
▶	6	4

Q26. Explain each letter of ACID:

- A – Atomicity
- C – Consistency
- I – Isolation
- D – Durability

Give a real-world example (e.g., bank transfer) showing why each property is important.

Atomicity

A transaction is completed fully or not at all.

Example: If money is deducted from one bank account but not credited to another, the transaction is rolled back.

Consistency

The database always remains in a valid state before and after a transaction.

Example: The total amount of money in the bank remains correct after a transfer.

Isolation

Multiple transactions do not interfere with each other.

Example: Two users transferring money at the same time do not affect each other's transactions.

Durability

Once a transaction is committed, the changes are permanently saved.

Example: Even if the server crashes after a successful bank transfer, the transferred amount remains updated.

Q27. Write a SQL transaction that does the following atomically:

1. Insert a new order (order_id=1011, customer_id=102, today's date, 'Pending', 1598.00)
2. Insert two order items for that order
3. Update the stock_qty of the purchased products
4. If any step fails, ROLLBACK the entire transaction. Otherwise, COMMIT.

Write the complete BEGIN...COMMIT/ROLLBACK block.

```
START TRANSACTION;
```

```
INSERT INTO orders  
VALUES  
(1011,102,CURDATE(),'Pending',1598.00);
```

```
INSERT INTO order_items  
VALUES  
(5016,1011,206,1,1299.00,0);
```

```
INSERT INTO order_items  
VALUES  
(5017,1011,208,1,599.00,0);
```

```
UPDATE products  
SET stock_qty = stock_qty - 1  
WHERE product_id = 206;
```

```
UPDATE products  
SET stock_qty = stock_qty - 1  
WHERE product_id = 208;
```

```
COMMIT;
```

```
-----END-----
```